

Module 03 (10 Marks)

Q1) Briefly explain switch statement. Write a c program to implement simple calculator using switch?

Ans: Switch:

The switch statement is a multiple way selector statement. There will be several alternatives; one of them will be executed based on the value of the expression.

Syntax:

```
switch (expression)
{
    case 1: statement-1;
        break;

    case 2: statement-2;
        break;

    case 3: statement-3;
        break;

    default: statement-4;
        break;
}
```

There are multiple cases, each case having its own set of statements.

Before switching to a particular case, the expression is tested.

If the result of the expression is 1, then case 1 will be executed.

If the result of the expression is 2, case 2 will be executed, and so on.

If none of the cases match the value of the expression, then the default case will be executed.

Ex: Write a C program to make simple calculations

```
#include <stdio.h>

void main()
{
    int a, b, ans, opn;
    clrscr();

    printf("Enter the value of a and b\n");
    scanf("%d %d", &a, &b);

    printf("Press 1 for +\n");
    printf("Press 2 for -\n");
    printf("Press 3 for *\n");
    printf("Press 4 for /\n");

    printf("Enter your option");
    scanf("%d", &opn);

    switch (opn)
    {
        case 1: ans = a + b;
            printf("Addition = %d", ans);
            break;
```

```

case 2: ans = a - b;
        printf("Subtraction = %d", ans);
        break;

case 3: ans = a * b;
        printf("Multiplication = %d", ans);
        break;

case 4: ans = a / b;
        printf("Division = %d", ans);
        break;

default: printf("Invalid option");
        break;
}

getch();
}

```

Q2) Describing the concept of leap year, write a C program to check whether the given year is leap year or not?

Ans: A leap year is a year that has 366 days instead of 365, with an extra day added to the month of February (29 days).

A year is a leap year if:

- It is divisible by 4,
- But century years (like 1900) must also be divisible by 400 to be a leap year.

Example: 2020 is a leap year, but 1900 is not; 2000 is a leap year.

```
#include <stdio.h>
```

```
void main ()
```

```

{
    int year;
    clrscr ();
    printf ("Enter year");
    scanf ("%d", &year);
    if (year % 4 == 0)
    {
        if (year % 100 == 0)
        {
            if (year % 400 == 0)
                printf ("Year is leap year");
            else
                printf ("Year is not leap year");
        }
    }
}

```

```

else
    printf("Year is leap year");
}
else
    printf("Year is not leap year");
getch ();
}

```

Q3) Write a C program to create an array of 10 floating point numbers and find the largest value among them?

Ans: #include <stdio.h>

```

void main ()
{
    float a [30], large;
    int n = 10, i;
    clrscr ();
    printf("Enter three elements");
    for (i = 0; i < n; i++)
    {
        scanf ("%f", &a[i]);
    }
    large = a[0];
    for (i = 1; i < n; i++)
    {
        if (a[i] > large)
        {
            large = a[i];
        }
    }
    printf("The largest among them = %f", large);
    getch ();
}

```

Q4) Write a c program to implement addition of two matrix -10 or sum of two matrixe?

Ans: #include <stdio.h>

void main ()

```
{
    int A [10][10], B [10][10], C [10][10];
    int i, j, n, m;
    clrscr ();
    printf ("Enter the value of n and m: ");
    scanf ("%d %d", &n, &m);
    printf ("Enter the elements of matrix A");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            scanf ("%d", &A[i][j]);
        }
    }
    printf ("Enter the elements of matrix B:");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            scanf ("%d", &B[i][j]);
        }
    }
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
        {
            C[i][j] = A[i][j] + B[i][j];
        }
    }
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++)
```

```

{
    printf("The addition of two matrix is /n");
    for (i=0; i<n, i++)
    {
        Printf ("%f", c [i] [j]);
    }
    getch ();
}

```

Q5) Explain the concept of call by reference. Write a C program to swapping of two numbers using call by reference? (From ChatGPT)

Ans: Call by Reference (Concept)

Call by reference is a method of passing arguments to a function in which the address of variables is passed instead of their actual values.

- In this method, the function receives pointers (addresses) of variables.
- Any changes made inside the function directly affect the original variables.
- It is useful when we want to modify actual values or return multiple values from a function.

C Program: Swapping of Two Numbers using Call by Reference

```
#include <stdio.h>
```

```
// Function declaration
```

```
void swap (int *x, int *y);
```

```
void main ()
```

```

{
    int a, b;
    printf("Enter two numbers:\n");
    scanf("%d %d", &a, &b);
    printf("Before swapping: a = %d, b = %d\n", a, b);

```

```
// Passing addresses
```

```

swap(&a, &b);
printf("After swapping: a = %d, b = %d\n", a, b);

```

```
}
```

```
// Function definition
```

```
void swap(int *x, int *y)
```

```

{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

```

Module 2 (10 Marks)

Q1) Write a short on

a. Array with its declaration and initialization

b. String with its declaration and initialization

Q1) Write a short note on array with its declaration and initialization

Defn: An array is collection of elements of same data type which are stored in the sequential order

Declaration of array

An array must be declared before it is used in the C-program. Array declaration can be made using appropriate data type

Syntax:

Data type Array Name [size];

example

```
int a [5];  
float b [20];
```

initialization of array

The process of assigning a value to an array is called array initialization

like initialization of an ordinary variable, the individual value of an array can be initialized

Syntax

Data type Array Name [size] =
{ val1, val2, val3, ..., valn };

example: `int a[5] = { 10, 20, 30, 40, 50 };`

* initialization without size

~~Array initialization~~

example: `int a[] = { 10, 20, 30, 40, 50 };`

Partial array initialization

example: `int a[5] = { 10, 20 };`

write a short note on declaration and initialization of string

Defn: A string is defined as an array of characters or sequence of characters enclosed within the double quote (" ") is called string

Declaring a string like int, float, double we don't have separate datatype for string in C. Therefore strings are treated as arrays of characters we use the data type char to declare a string variable

Syntax

char Data type . String Name [size];

example

char b[20];

In this case b is a string which can hold 20 characters including null character [0]
NULL

Initialization of String

The process of assigning of character is called String initialization we can do initialization in two type.

1. initializing character by character
2. initializing string directly

1. character by character

In this case individual character to be initialized is ^{written} ~~written~~ within the single quote and must be separated by a comma. The NULL character should be written in a single quote at the end of the string.

example `char a[] = {'k', 'b', 'n', 'u', '\0'};`

2. initializing string directly

The string is directly assigned to a string variable by writing sequence of character within the double quote.

example = `char a[50] = "kbnv";`

Q2) Briefly explain categories of function with suitable example? (From ChatGPT, Refer programmes from notes)

Ans: Categories of Functions in C

In C, functions are categorized based on arguments (parameters) and return values.

1. Function with no arguments and no return value

- Does not take any input from the calling function.
- Does not return any value.
- Performs operations internally.

Example:

```
#include <stdio.h>

void display()
{
    printf("Hello World");
}
void main()
{
    display();
}
```

2. Function with arguments and no return value

- Takes input (arguments) from the calling function.
- Does not return any value.

Example:

```
#include <stdio.h>
void add(int a, int b)
{
    int sum;
    sum = a + b;
    printf("Sum = %d", sum);
}
void main()
{
    add(5, 3);
}
```

3. Function with no arguments but returns a value

- Does not take input from the calling function.
- Returns a value to the calling function.

Example:

```
#include <stdio.h>

int add()
{
    int a = 5, b = 3;
    return (a + b);
}
```

```
void main()
{
    int result;
    result = add();
    printf("Sum = %d", result);
}
```

4. Function with arguments and returns a value

- Takes input (arguments).
- Returns a value to the calling function.
- Most commonly used type.

Example:

```
#include <stdio.h>
```

```
int add(int a, int b)
{
    return (a + b);
}
```

```
void main()
{
    int result;
    result = add(5, 3);
    printf("Sum = %d", result);
}
```

Conclusion

Functions in C are classified into four categories based on arguments and return values, which helps in writing modular and reusable programs.

Q3) Briefly explain components of user defined function?

Ans: Components of User Defined Function in C

A user-defined function is a function created by the programmer to perform a specific task. It has the following main components:

1. Function Declaration (Prototype)

- It tells the compiler about the function name, return type, and parameters before its actual definition.
- It helps in type checking.

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int);
```

2. Function Definition

- It contains the actual body of the function.
- Specifies what the function does.

Syntax:

```
return_type function_name(parameter_list)
{
    // statements
}
```

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

3. Function Call

- It is used to invoke (execute) the function.
- Control is transferred to the function when it is called.

Example:

```
int result;
result = add(5, 3);
```

4. Parameters (Arguments)

- Actual Parameters: Values passed in function call.
- Formal Parameters: Variables used in function definition.

Example:

```
add(5, 3); // Actual parameters
int add(int a, int b) // Formal parameters
```

5. Return Statement

- Used to return a value from function to calling function.
- Terminates the function execution.

Example:

```
return a + b;
```

Complete Example Program

```
#include <stdio.h>
```

```
// Function declaration  
int add(int, int);
```

```
void main()  
{  
    int result;
```

```
    // Function call  
    result = add(4, 6);
```

```
    printf("Sum = %d", result);  
}
```

```
// Function definition  
int add(int a, int b)  
{  
    return a + b;  
}
```

Conclusion

User-defined functions improve modularity, readability, and reusability of programs by dividing a large problem into smaller parts.

Q4) Define recursion. Write a C program to find factorial of a given number using recursion.
(From ChatGPT)

Ans: Recursion (Definition)

Recursion is a process in which a function calls itself repeatedly until a base condition is satisfied.

- The function keeps calling itself with a smaller input.
- It stops when the base case is reached.
- Useful for problems that can be broken into smaller sub-problems.

C Program to Find Factorial Using Recursion

Write a C program to find the factorial of a number using a recursive function.

```
#include <stdio.h>
void main()
{
    int num, fact;
    int factorial (int n); /* Function declaration */

    printf("Enter a number\n");
    scanf("%d", &num);

    fact = factorial (num);
    printf("Factorial of %d = %d\n", num, fact);

} /* End of main() */
```

191

CS CamScanner

Programming in C and Data Structures

```
/* Recursive function for factorial */
int factorial (int n)
{
    int fct;

    if(n == 0) /* stopping condition */
    {
        return(1);
    }
    else
    {
        fct = n * factorial ( n - 1 ); /* recursive call */
    }

    return (fct);
} /* End of factorial() */
```

Output

Run1

Enter a number

5

Factorial of 5 = 120

Run2

Enter a number

10

Factorial of 10 = 24320

SUMMARY